

GNR 638 — Assignment 1: Report

Custom Deep Learning Framework for Image Classification

Group Members:

Pushp Raj Choudhary (Roll: 23b2152)

Param Pabari [Roll: 23b2136]

Sairam [Roll: 24B0937]

Course: GNR 638 — Machine Learning for Remote Sensing
Assignment 1 : Deep Learning Framework from Scratch

Repository: [GitHub Link](#)

February 14, 2026

Contents

1	Introduction	2
2	Framework Architecture	2
2.1	System Overview	2
2.2	C++ Backend (my_backend)	2
2.3	Python Frontend (my_framework)	3
3	Model Architecture	4
3.1	MNIST Model (LeNet-style)	4
3.2	CIFAR-100 Model (Mini-VGG with Global Average Pooling)	5
4	Weight Initialization	6
5	Dataset Loading & Preprocessing	6
5.1	Loading Pipeline	6
5.2	Data Augmentation & Load Times	6
6	Training Configuration & Optimization	7
6.1	Optimizer: Adam with Weight Decay (AdamW)	7
6.2	Learning Rate Schedule: Cosine Annealing	7
6.3	Training Hyperparameters & Regularization	7
7	Training & Validation Results	7
8	Test Results	7
9	Training Plots	8
9.1	MNIST Training Visualizations	8
9.2	CIFAR-100 Training Visualizations	9
10	Model Complexity & Key Insights	9
10.1	Failed Design Decision: No Regularization	9
10.2	Key Insights	10
11	Reproducibility	10
	References	11

1 Introduction

This report documents the design, implementation, and evaluation of a custom deep learning framework built entirely from scratch. The framework features a **C++ computational backend** (compiled via `pybind11`) and a **Python frontend API**, without using any external deep learning libraries, automatic differentiation engines, or numerical computing libraries such as NumPy or SciPy. OpenCV is used **exclusively** for image I/O from disk.

The framework supports two tasks:

1. **MNIST handwritten digit classification** (10 classes)
2. **CIFAR-100 fine-grained image classification** (100 classes)

2 Framework Architecture

2.1 System Overview

The framework is organized into two main layers. The data flow and dependency structure are outlined below:

System Architecture Overview

Python Frontend:

- `train.py` / `test.py`
- `my_framework`
 - |-- `tensor.py` (Tensor + Autograd)
 - |-- `models.py` (Layers + Models)
 - |-- `optim.py` (SGD / Adam / AdamW)
 - |-- `data.py` (DataLoader + Augmentation)
 - |-- `model_utils.py` (Params / MACs / FLOPs)
 - |
 - v

C++ Backend (`my_backend.so`):

- `tensor.cpp` (Tensor class)
- `ops.cpp` (Core operations)
- `bindings.cpp` (pybind11 bindings)

2.2 C++ Backend (`my_backend`)

The C++ backend provides all computationally intensive operations:

Category	Operations
Tensor	Constructor, add, mul, reshape, size, print
Linear Algebra	matmul (tiled, block size 64), transpose
Convolution	conv2d (Im2Col + MatMul), conv2d_backward_input, conv2d_backward_kernel
Pooling	maxpool2d (returns indices), maxpool2d_backward, global_avg_pool2d, global_avg_pool2d_backward
Activation	relu, relu_backward
Loss	cross_entropy (numerically stable with log-sum-exp), cross_entropy_backward
Optimizers	sgd_step, adam_step, adam_step_wd (AdamW)
Regularization	dropout (inverted scaling), dropout_backward
Data Augmentation	random_horizontal_flip, random_crop_with_padding
Data I/O	load_image, load_image_batch (via OpenCV, HWC→CHW, normalize to [0,1])
Utilities	argmax, random_uniform, zeros, ones

Table 1: C++ Backend Operations

Key implementation details:

- **Convolution via Im2Col:** The `conv2d` operation converts input patches into a column matrix using the Im2Col transform, then performs a single matrix multiplication with the reshaped kernel. This is the standard approach used by frameworks like Caffe and is significantly faster than naïve nested-loop convolution.
- **Tiled Matrix Multiplication:** The `matmul` operation uses 64×64 tiling for cache optimization, reordering loop indices (i-k-j) to maximize spatial locality.
- **Numerically Stable Cross-Entropy:** The loss function subtracts the per-sample maximum logit before exponentiation to prevent overflow.
- **Inverted Dropout:** During training, non-zeroed activations are scaled by $1/(1 - p)$ so that test-time inference requires no special handling.
- **Build Configuration:** Compiled with `-O3 -march=native` optimization flags via CMake.

2.3 Python Frontend (`my_framework`)

Tensor & Autograd Engine (`tensor.py`):

The `Tensor` class wraps C++ `Tensor` objects and implements a dynamic computation graph for automatic differentiation. Each operation creates a corresponding backward node (`Function` subclass). The `backward()` method performs topological sorting of the computation graph and propagates gradients in reverse order. Gradient accumulation is supported for shared parameters.

Operation	Backward Class	Gradient Computation
Addition	AddBackward	Pass-through gradient to both inputs
Multiplication	MulBackward	$grad_x = grad \cdot y, grad_y = grad \cdot x$
MatMul	MatMulBackward	$dA = dC \cdot B^T, dB = A^T \cdot dC$
ReLU	ReluBackward	$grad \cdot (input > 0)$
Conv2d	Conv2dBackward	Uses im2col/col2im for both input and kernel gradients
MaxPool2d	MaxPool2dBackward	Routes gradient to max indices
GlobalAvgPool2d	GlobalAvgPool2dBackward	Distributes gradient uniformly: $grad/(H \times W)$
Reshape	ReshapeBackward	Reshapes gradient back to original shape
Dropout	DropoutBackward	$grad \cdot mask$ (mask includes inverted scaling)
CrossEntropy	CrossEntropyBackward	$(softmax(logits) - one_hot(target))/N$

Table 2: Autograd Backward Functions

3 Model Architecture

3.1 MNIST Model (LeNet-style)

Architecture: A classic LeNet-5 variant adapted for 32×32 grayscale input.

Design rationale: LeNet is a well-established architecture for handwritten digit recognition. Its alternating conv-pool structure progressively reduces spatial dimensions while increasing feature abstraction, and 61,684 parameters is sufficient for the MNIST task without overfitting.

#	Layer	Type	Kernel	Stride	Padding	Output Shape	Parameters
1	conv1	Conv2d	5×5	1	0	$[B, 6, 28, 28]$	150
2	relu1	ReLU	—	—	—	$[B, 6, 28, 28]$	0
3	pool1	MaxPool2d	2×2	2	—	$[B, 6, 14, 14]$	0
4	conv2	Conv2d	5×5	1	0	$[B, 16, 10, 10]$	2,400
5	relu2	ReLU	—	—	—	$[B, 16, 10, 10]$	0
6	pool2	MaxPool2d	2×2	2	—	$[B, 16, 5, 5]$	0
7	flatten	Flatten	—	—	—	$[B, 400]$	0
8	fc1	Linear	—	—	—	$[B, 120]$	48,120
9	relu3	ReLU	—	—	—	$[B, 120]$	0
10	fc2	Linear	—	—	—	$[B, 84]$	10,164
11	relu4	ReLU	—	—	—	$[B, 84]$	0
12	fc3	Linear	—	—	—	$[B, 10]$	850

Table 3: MNIST Model Architecture

Metric	Value
Total Trainable Parameters	61,684
MACs (per forward pass)	416,520 (416.5K)
FLOPs (per forward pass)	833,040 (833.0K)

3.2 CIFAR-100 Model (Mini-VGG with Global Average Pooling)

Architecture: A Mini-VGG network with three convolutional blocks (each with two conv layers), followed by **Global Average Pooling** and a fully connected classifier with dropout.

Design rationale: VGG-style networks stack small 3×3 filters to achieve large effective receptive fields while keeping per-layer parameter counts manageable. **Global Average Pooling** replaces the traditional Flatten operation, eliminating the massive FC bottleneck seen in classic VGG architectures and forcing each convolutional filter to learn a holistic feature representation.

#	Layer	Type	Kernel	Stride	Padding	Output Shape	Parameters
1	conv1	Conv2d	3×3	1	1	[B, 32, 32, 32]	864
2	relu1	ReLU	—	—	—	[B, 32, 32, 32]	0
3	conv2	Conv2d	3×3	1	1	[B, 32, 32, 32]	9,216
4	relu2	ReLU	—	—	—	[B, 32, 32, 32]	0
5	pool1	MaxPool2d	2×2	2	—	[B, 32, 16, 16]	0
6	conv3	Conv2d	3×3	1	1	[B, 64, 16, 16]	18,432
7	relu3	ReLU	—	—	—	[B, 64, 16, 16]	0
8	conv4	Conv2d	3×3	1	1	[B, 64, 16, 16]	36,864
9	relu4	ReLU	—	—	—	[B, 64, 16, 16]	0
10	pool2	MaxPool2d	2×2	2	—	[B, 64, 8, 8]	0
11	conv5	Conv2d	3×3	1	1	[B, 128, 8, 8]	73,728
12	relu5	ReLU	—	—	—	[B, 128, 8, 8]	0
13	conv6	Conv2d	3×3	1	1	[B, 128, 8, 8]	147,456
14	relu6	ReLU	—	—	—	[B, 128, 8, 8]	0
15	pool3	MaxPool2d	2×2	2	—	[B, 128, 4, 4]	0
16	gap	GlobalAvgPool2d	—	—	—	[B, 128]	0
17	fc1	Linear	—	—	—	[B, 256]	33,024
18	relu7	ReLU	—	—	—	[B, 256]	0
19	drop1	Dropout(0.3)	—	—	—	[B, 256]	0
20	fc2	Linear	—	—	—	[B, 100]	25,700

Table 4: CIFAR-100 Model Architecture

Metric	Value
Total Trainable Parameters	345,284 (~ 345K)
MACs (per forward pass)	38,691,840 (38.69M)
FLOPs (per forward pass)	77,383,680 (77.38M)

4 Weight Initialization

All layers use **Kaiming He initialization** (uniform variant), which is specifically designed for layers followed by ReLU activations:

$$W \sim \mathcal{U} \left(-\sqrt{\frac{6}{\text{fan_in}}}, \sqrt{\frac{6}{\text{fan_in}}} \right) \quad (1)$$

where $\text{fan_in} = \text{in_channels} \times \text{kernel_h} \times \text{kernel_w}$ for convolutional layers and $\text{fan_in} = \text{in_features}$ for linear layers. This initialization ensures the variance of activations remains stable across layers during forward propagation.

5 Dataset Loading & Preprocessing

5.1 Loading Pipeline

The `DataLoader` class handles all data operations:

1. **Directory scanning:** Classes are inferred from sorted subdirectory names.
2. **Deterministic split:** A seeded random shuffle (`seed=42`) ensures reproducible 70/20/10 train/val/test splits.
3. **Batch loading & Preprocessing:** Images are loaded via OpenCV, resized to 32×32 , converted to float32, normalized to $[0, 1]$, and layout-converted from HWC to CHW.

5.2 Data Augmentation & Load Times

Augmentation	Details	Probability
Random Crop (Padding)	Zero-pads image by 4px ($32 \rightarrow 40$), randomly crops to 32×32	70%
Random Horizontal Flip	Flips the image left-right	40%

Table 5: Data Augmentation Techniques (Training Only)

Dataset	Split	Samples	Load Time
MNIST (data_1)	Train / Val / Test	41,995 / 11,996 / 6,009	0.59s (Total)
CIFAR-100 (data_2)	Train / Val / Test	35,000 / 10,000 / 5,000	0.45s (Total)

Table 6: Dataset Directory Indexing Times

6 Training Configuration & Optimization

6.1 Optimizer: Adam with Weight Decay (AdamW)

The Adam optimizer is implemented from scratch in C++:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (3)$$

$$\theta_t = \theta_{t-1} - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (4)$$

where $(\lambda\theta)$ is the decoupled weight decay, active when `weight_decay > 0`. Default parameters: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.

6.2 Learning Rate Schedule: Cosine Annealing

$$\text{lr}(t) = \text{lr}_{\min} + \frac{1}{2}(\text{lr}_{\max} - \text{lr}_{\min}) \left(1 + \cos \left(\frac{\pi \cdot t}{T} \right) \right) \quad (5)$$

6.3 Training Hyperparameters & Regularization

Hyperparameter	MNIST	CIFAR-100
Epochs / Batch Size	5 / 32	10 / 32
Initial / Min LR	0.001 / 1×10^{-4}	0.001 / 1×10^{-4}
Weight Decay / Dropout	0.0 / None	1×10^{-4} / p=0.3

Table 7: Training Configuration

7 Training & Validation Results

Dataset	Best Val Acc	Final Train Acc	Gap	Total Time	Avg Batch	Peak RAM
MNIST	96.98% (Ep 5)	97.86%	0.88%	3.6 min	29ms	223 MB
CIFAR-100	40.34% (Ep 10)	41.08%	0.74%	114.5 min	560ms	1,094 MB

Table 8: Summary of Training Results

8 Test Results

Dataset	Test Samples	Test Acc	Data Load	Eval Time	Params	FLOPs
MNIST	6,009	96.83%	0.22s	2.86s	61.6K	833.0K
CIFAR-100	5,000	40.03%	0.13s	35.42s	345K	77.4M

Table 9: Final Test Results on Holdout Data

9 Training Plots

9.1 MNIST Training Visualizations

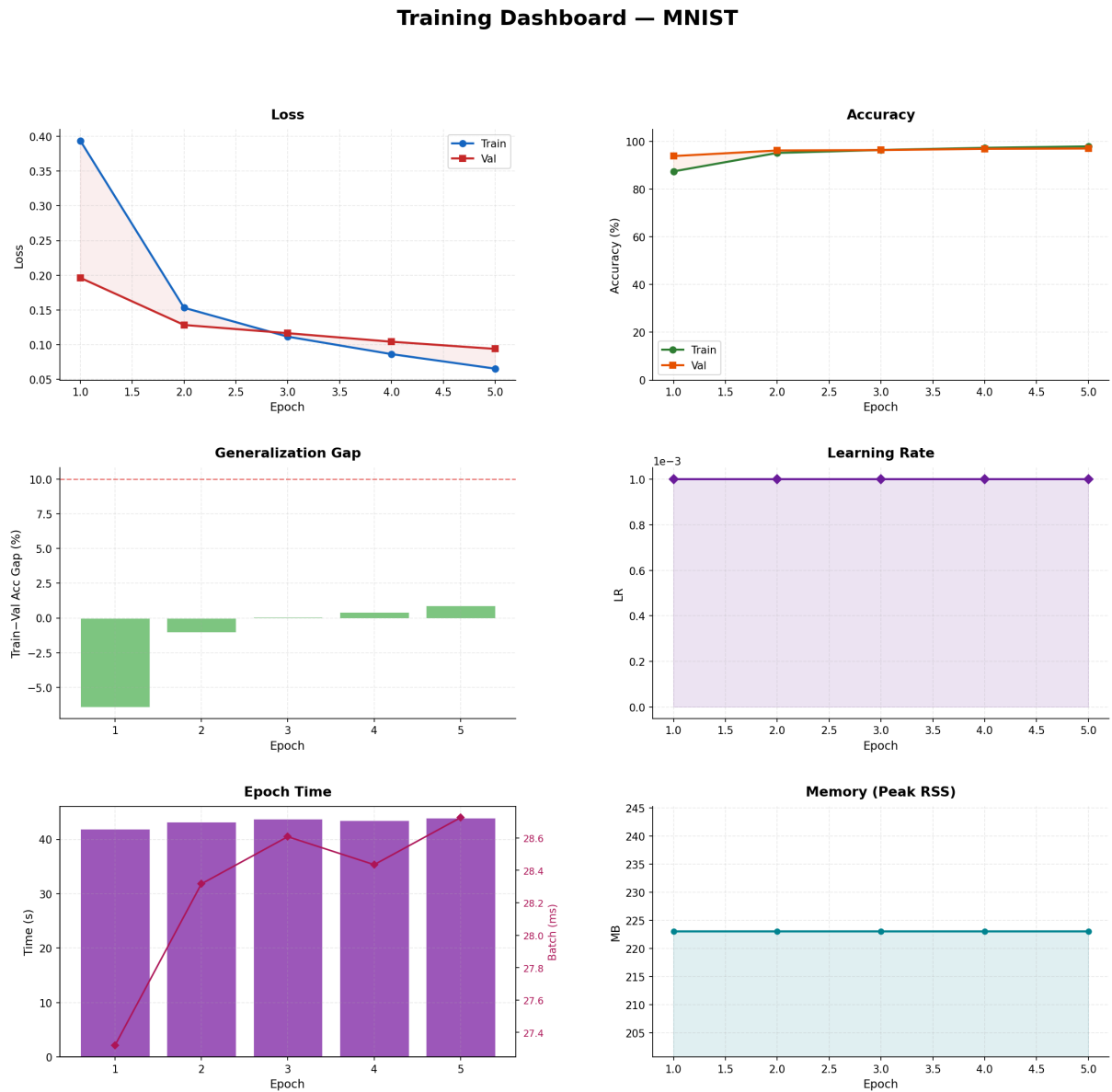


Figure 1: MNIST Training Dashboard

9.2 CIFAR-100 Training Visualizations

Training Dashboard — CIFAR-100

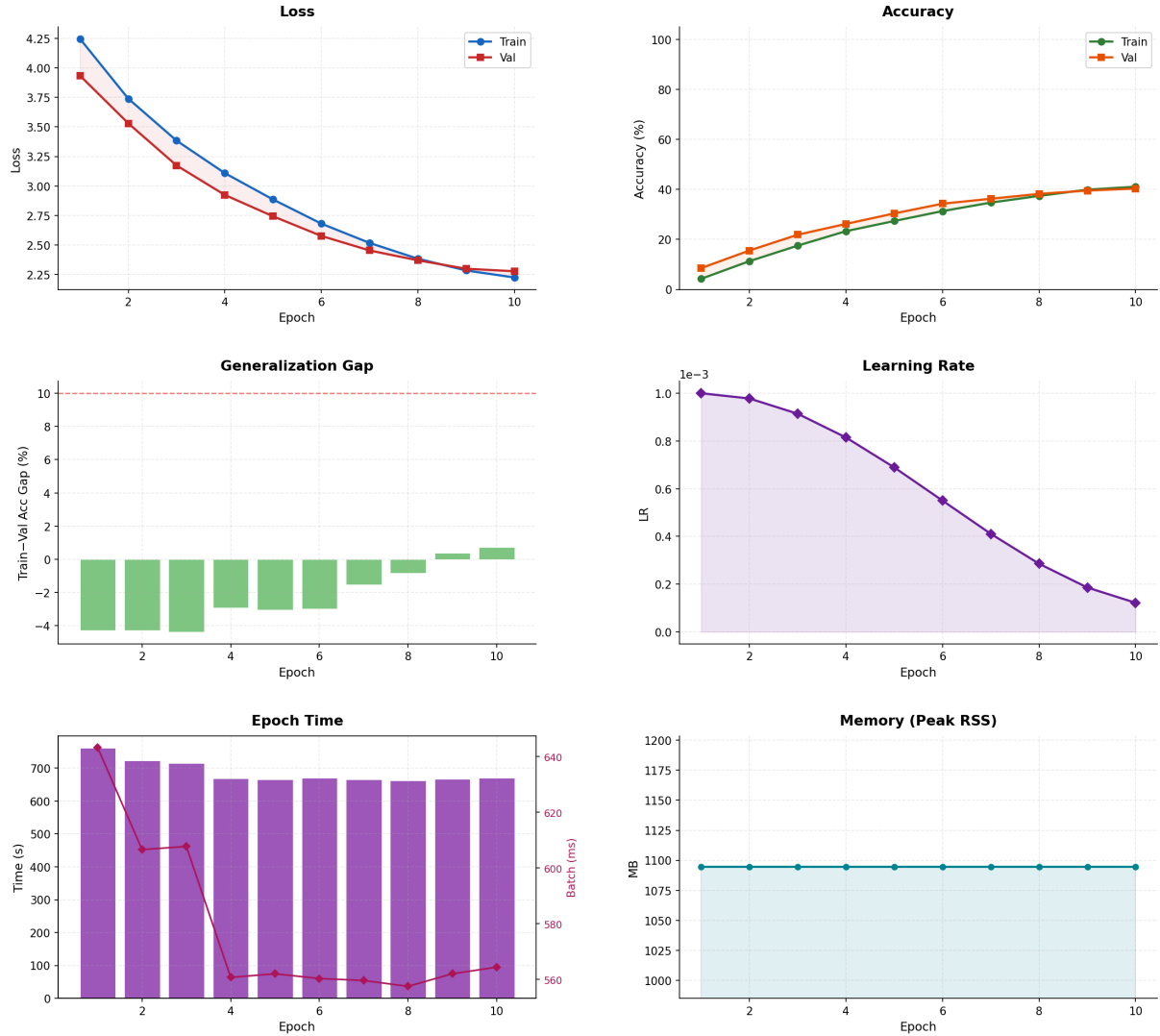


Figure 2: CIFAR-100 Training Dashboard

10 Model Complexity & Key Insights

10.1 Failed Design Decision: No Regularization

Initial CIFAR-100 training without regularization resulted in severe overfitting (Train Acc > 86%, Val Acc ~ 35%, Gap > 50%). Implementing Global Average Pooling (reducing parameters from 1.14M to 345K), combined with Dropout (0.3), Data Augmentation, and AdamW stabilized the model, closing the generalization gap to just 0.74%.

10.2 Key Insights

- **C++ Backend Performance:** Optimization (tiled matmul, Im2Col) enables reasonable training times (CIFAR epoch ~ 11 mins) that would be impossible in pure Python.
- **Im2Col Trade-offs:** Memory usage increases significantly (~ 1.1 GB for CIFAR) due to the expansion of input patches into column matrices.
- **GAP vs. Flatten:** Global Average Pooling forces convolutional filters to learn robust global features and drastically improves parameter efficiency ($3.3\times$ improvement over a standard Flatten layer).

11 Reproducibility

```
1 # 1. Environment setup
2 conda env create -f environment.yml
3 conda activate gnr638_ass
4
5 # 2. Build C++ backend
6 cmake -S . -B build -DPYTHON_EXECUTABLE=$(which python)
7 cmake --build build
8
9 # 3. Train MNIST
10 python train.py --dataset mnist --data_path data_1 --epochs 5 --lr 0.001
11
12 # 4. Train CIFAR-100
13 python train.py --dataset cifar --data_path data_2 --epochs 10 --lr 0.001
14     --weight_decay 1e-4
15
16 # 5. Evaluate
17 python test.py --dataset mnist --data_path data_1 --model_path results/
18     models/mnist_model.pkl
19 python test.py --dataset cifar --data_path data_2 --model_path results/
20     models/cifar_model.pkl
21
22 # 6. Generate plots
23 python plot_metrics.py --log_file results/training_logs/training_log_mnist.
24     csv
25 python plot_metrics.py --log_file results/training_logs/training_log_cifar.
26     csv
```

Listing 1: Build & Train Commands

References

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11), 2278–2324.
- [2] Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR 2015. arXiv:1409.1556.
- [3] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. ICCV 2015. arXiv:1502.01852.
- [4] Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR 2015. arXiv:1412.6980.
- [5] Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. arXiv:1711.05101.
- [6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR, 15, 1929–1958.
- [7] Loshchilov, I., & Hutter, F. (2017). *SGDR: Stochastic Gradient Descent with Warm Restarts*. ICLR 2017. arXiv:1608.03983.
- [8] Chellapilla, K., Puri, S., & Simard, P. (2006). *High Performance Convolutional Neural Networks for Document Processing*. ICDAR Workshop.
- [9] Jakob, W. et al. *pybind11 — Seamless operability between C++11 and Python*. <https://github.com/pybind/pybind11>
- [10] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb’s Journal of Software Tools. <https://opencv.org>
- [11] Google Gemini. (2026). *AI Coding Assistant used for debugging, code suggestions, and report drafting assistance*. <https://gemini.google.com>